Web Server Log Processing using Hadoop in Azure

Business Overview

A comprehensive data pipeline is essential for the efficient extraction, processing, and analysis of log data, which plays a crucial role in evaluating performance and identifying operational trends. In today's data-driven environment, leveraging scalable technologies like Hadoop, Hive, Spark, and Flume allows organizations to handle large volumes of log data seamlessly. These tools work together to ensure that data is ingested from various sources, stored in a distributed manner, and processed efficiently for actionable insights. By implementing such a pipeline, businesses can gain valuable insights from their log data to optimize operations, enhance customer experiences, and drive data-driven decision-making.

Aim:

The project focuses on processing and analyzing NASA's raw log data to extract valuable insights into server activity, user behavior, and operational trends. The primary objective is to transform and standardize the data through a scalable pipeline. Raw log data is ingested into Hadoop's HDFS, ensuring that it is stored in a distributed and fault-tolerant manner. Data processing is orchestrated using Spark and Hive, allowing for efficient querying and transformation. The project aims to uncover patterns in user interactions, server performance, and other critical operational metrics.

Tech Stack:

- **Programming:** SQL, Scala
- **Services:** Azure VM, PuTTy, Hadoop, Hive, Flume, Scala, Spark

Azure VM :

Azure VMs are essentially virtualized computing environments that mimic physical computers. They come equipped with CPU, memory, storage, and networking capabilities, allowing users to install operating systems and applications as needed[2][4]. This flexibility enables organizations to deploy applications quickly and efficiently while managing costs effectively.

Hadoop:

Hadoop is an open-source framework that allows you to store and process large amounts of data in a distributed way. It breaks down data into smaller pieces and stores them across many computers, making it possible to handle big data even if a single computer can't manage it. It uses multiple machines working together to process the data. Hadoop is a framework designed to handle large-scale data processing across many machines. It includes two key components: **HDFS (Hadoop Distributed File System)** and **YARN (Yet Another Resource Negotiator)**.

Hive:

Hive is a data warehouse system that is used to analyze structured data. It is built on the top of Hadoop. It was developed by Facebook. Hive provides the functionality of reading, writing, and managing large datasets residing in distributed storage. It runs SQL-like queries called HQL (Hive query language) which gets internally converted to MapReduce jobs.

Using Hive, we can skip the requirement of the traditional approach of writing complex MapReduce programs. Hive supports Data Definition Language (DDL), Data Manipulation Language (DML), and User Defined Functions (UDF).

Flume:

Apache Flume is an open-source, distributed service used to collect, aggregate, and move large amounts of log data and other event-based data into Hadoop's HDFS (Hadoop Distributed File System) or other data stores for processing and analysis. It is designed to handle high-throughput, low-latency data flows, and is commonly used to stream log data from various sources into Hadoop for storage and processing.

The main components of Flume include:

- **Source**: The point where data enters Flume, such as a log file, a network socket, or a custom source.
- **Channel**: The medium through which the data flows after being collected by the source. Flume supports different types of channels like memory or file-based.
- **Sink**: The destination where the data is sent after being passed through the channel, such as HDFS or a database.

Spark:

Apache Spark is an open-source, distributed computing system that allows you to process large volumes of data quickly. It provides an interface for programming entire

clusters with implicit data parallelism and fault tolerance. Spark is designed to be faster and more flexible than Hadoop, as it processes data in memory, avoiding the time-consuming disk-based storage methods.

Scala:

Scala is a high-level programming language often used with Apache Spark. It combines the best features of object-oriented and functional programming. Scala runs on the Java Virtual Machine (JVM) and is fully interoperable with Java. Its concise syntax, high performance, and support for functional programming make it a popular choice for Spark.

Data Description:

This project uses a dataset consisting of log entries captured by the NASA Gatekeeper server, providing insights into HTTP requests made by clients. The logs are stored in an ASCII file format, with one line representing each request. The key components of each log entry are:

1. **Host**: The hostname or Internet address of the client making the request, which identifies the source of the request.
2. **Timestamp**: The date and time when the request was made, formatted as "DAY MON DD HH:MM:SS YYYY". The timestamp includes the day of the week, the month, the day of the month, the time in 24-hour format, the year, and the timezone (-0400).
3. **Request**: The specific HTTP request made by the client, encapsulated in quotes. This includes the requested file or resource.
4. **HTTP Reply Code**: The HTTP status code returned by the server, indicating whether the request was successful or if there were errors (e.g., 200 for success, 304 for not modified).
5. **Bytes in Reply**: The number of bytes sent in response to the request. This represents the size of the returned data.

Approach:

1. A virtual machine was created in Azure with an optimized configuration for low computation costs and high efficiency.

2. The PuTTY application was installed to establish a secure SSH connection between the local machine and the Azure VM. The public IP of the Azure VM was specified in PuTTY to enable remote access.
3. The disk allocated during VM creation was mounted to ensure persistent storage across reboots. This helps separate system files from application data, improving management and performance.
4. Frameworks like Hadoop were installed on the Azure VM using specific commands to set up a distributed data processing environment. The configuration ensured Hadoop was ready for large dataset handling and job execution.
5. Hive and Flume were installed using the commands in the codes section to process and transport log data efficiently. Detailed commands were executed for both, with logs created for monitoring the setup and execution process.
6. Apache Spark and Scala were installed on the VM to enable distributed data processing. Spark was configured to submit and run jobs, with Scala as the programming language for Spark interactions.
7. A Hive table was created to store NASA logs and run queries on them. The dataset was loaded into the table, and execution logs were generated to track data ingestion and query performance.

Key Takeaways:

- Understanding the problem statement
- Overview of Microsoft Azure
- Creating a Virtual machine (VM) instance in Microsoft Azure.
- Connect to the Virtual machine using the Putty application
- Install various big data technologies on a Virtual machine
- Understanding log files and exploring their different types.
- Techniques for processing log files and the significance of log data analysis.
- Identifying the role of referrers and user agents in log files.
- Utilizing Flume for efficient log data ingestion and processing.
- Configuring Flume agents for seamless data flow and collection.
- Ingesting and processing log data using Flume for streamlined analytics.
- Processing log data with Spark to extract valuable insights.

Note: We recommend monitoring Azure Services and deleting those that are not in use, as they can result in high costs.

Architecture Diagram: